

A: ShufflingRobot

Source file name: shuffling.c, shuffling.cpp, shuffling.java, or shuffling.py
 Author: Rodrigo Cardoso

Joe Wiz, the famous magician, whose reputation comes from his surprising card tricks, has a new stage helper: SR, a robot that can shuffle decks. What is difficult to realize is that SR can shuffle in exactly the same way, any number of times, a given deck. It also has the ability to see the exact place in the deck a given card is located at. *trucos* *ayudante de escena* *baajar cartas.* *darse cuenta, comprender*

Joe has developed a trick that asks a person to pick a card (that Joe cannot see) and insert it again -face down- at some place of the deck. Then SR shuffles the deck some number of times. Finally, Joe asks the person to count some number of cards from the top of the deck in order to find the initially picked card. *baaja*

This trick can be played with any deck (e.g., different number of cards, suits, etc.). The way that SR uses to shuffle is changed for every performance. Why can this trick be successful?

It is clear that SR may detect where the given card is inserted and, with this information that Joe secretly acquires, it shuffles the deck a convenient number of times. This number of shuffles is such that the deck comes again exactly to the original deck ordering that appeared before the first shuffle. Then, it is clear that the given card can be found counting the number of cards from the top that was known at the beginning of the trick.

For example, let's suppose that the deck has only seven cards, say

$J\clubsuit, 4\spadesuit, 3\heartsuit, 4\diamondsuit, 5\heartsuit, A\heartsuit, 7\clubsuit$

and that SR shuffles giving the reordering

$7\clubsuit, 4\diamondsuit, J\clubsuit, 4\spadesuit, 5\heartsuit, A\heartsuit, 3\heartsuit$

If a shuffling is applied to the first one, the new arranging is:

$3\heartsuit, 4\spadesuit, 7\clubsuit, 4\diamondsuit, 5\heartsuit, A\heartsuit, J\clubsuit$

It should come as no surprise that after 6 shuffles, the original arrangement will be obtained. *disposición*

Given the way that SR uses to shuffle each time, Joe wants to detect the minimum number of shuffles to be performed to return a deck to its original arrangement. Can you help Joe?

Input

The input consists of several test cases. Each case is defined with two input lines of the form

N
 $a_1 a_2 \dots a_N$

The first line contains a positive integer number N , corresponding to the size of the deck, $1 < N < 53$. The second line establishes the deck reordering that SR applies every time it shuffles: the card at place i is moved to the place a_i . You may assume that the second line contains every integer number from 1 to N .

The end of the input is signaled with a line containing a single 0.

The input must be read from standard input.

Output

For each test case, output a line with an integer value corresponding to the minimum number of shuffles that SR must accomplish to attain the original deck arrangement.

2023 ACIS REDIS - XXXVII Colombian Programming Contest - ICPC

The output must be written to standard output.

Sample Input	Sample Output
7	6
3 4 7 2 5 6 1	0
3	2
1 2 3	
3	
2 1 3	
0	

B: Idoneal

Source file name: idoneal.c, idoneal.cpp, idoneal.java, or idoneal.py

Author: Rodrigo Cardoso

$$a(bc) + bc$$

Pick three positive integers $a < b < c$ and calculate $n = a \cdot b + a \cdot c + b \cdot c$. The result n is a *non idoneal* number. The *idoneal* numbers are those positive integers that cannot be obtained in this way. For example, it is easy to check that 5 is idoneal, but 11 is not because $1 \cdot 2 + 1 \cdot 3 + 2 \cdot 3 = 11$.

Your problem is simple: decide if a given positive integer N is idoneal. You are required to present numerical evidence to support negative answers.

Input

The input consists of several test cases. Each test case is defined by a single line containing a positive integer N ($1 < N < 1\,000\,000$). The end of the input is signaled with a line containing the number 0; this line should not be processed.

The input must be read from standard input.

Output

For each test case, output a single line with the text 'idoneal' if N is idoneal and three blank separated positive integers a, b, c , such that $a < b < c$ and $N = a \cdot b + a \cdot c + b \cdot c$, otherwise. In this latter case, if there are several triples satisfying the equation, favor the combination with maximum a ; if there are at least two of these solutions, then favor the triple with maximum b .

The output must be written to standard output.

Sample Input

10
11
16
17
18
0

Sample Output

idoneal
1 2 3
idoneal
1 2 5
idoneal

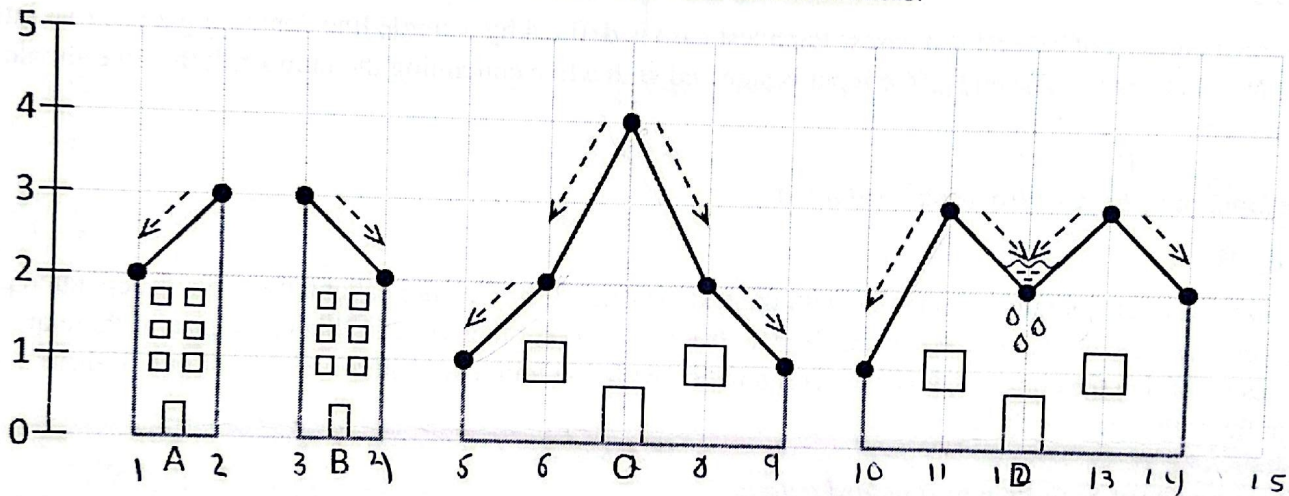
C: Water Drainage

Source file name: `water.c`, `water.cpp`, `water.java`, or `water.py`
 Author: Andrés Mejía

Architects like to use different materials, shapes and color palettes when designing a house.

An important part of a house is the roof, which protects the inside from the rain. Architects have scary degrees of freedom when designing the roof of a house. However, they must guarantee that water drains properly. If the roof is improperly designed, water will accumulate in low points of the roof and cause problems.

For the purpose of this problem, the roof of a house is a continuous line composed of one or more connected line segments. The following figure shows four houses with four different roofs:



The dashed arrows show the direction in which rain water flows after making contact with different sections of the roof. Houses A, B and C have properly designed roofs because —regardless of where the rain falls— the water is able to flow downwards and eventually falls off the roof on either side of the house.

On the other hand, house D has an improperly designed roof, because the water in the middle gets trapped and causes the roof to flood. This is a disaster because the extra weight could cause the entire roof to collapse!

Please keep everyone safe by writing a program that determines if the roof of a house is properly designed or not.

Input

The input consists of several test cases. Each test case describes a different roof and consists of multiple lines.

The first line of each test case contains an integer N ($2 \leq N \leq 1000$). Each of the following N lines contains a pair of integers X_i and Y_i separated by a single blank, for $1 \leq i \leq N$. These pairs of integers represent points on the Cartesian plane. The roof is formed by connecting these points (in the same order that they are given in the input) by straight line segments.

It is guaranteed that the points will be given left to right (in other words, $X_i < X_{i+1}$ for $1 \leq i < N$) and the roof won't contain any flat sections (in other words, $Y_i \neq Y_{i+1}$ for $1 \leq i < N$). It is also guaranteed that $1 \leq X_i \leq 2\,000\,000$ and $1 \leq Y_i \leq 2\,000\,000$ for $1 \leq i \leq N$.

The last line of the input contains a single 0 and should not be processed.

The input must be read from standard input.

Output

For each test case output the word **safe** if the roof is properly designed, or the word **disaster** if it's not. Keep in mind that the judging system is case-sensitive, so your output should be lowercase exactly as shown in the samples below.

The output must be written to standard output.

Sample Input	Sample Output
2	safe
1 2	safe
3 3	safe
2	disaster
1 3	
3 2	
5	
1 1	
2 2	
3 4	
4 2	
5 1	
5	
1 1	
2 3	
3 2	
4 3	
5 2	
0	